GCP vs Databricks

Platform Comparison Guide

Understanding Databricks in GCP Context

For: Data Engineers Transitioning to Databricks

INTRODUCTION: What is Databricks?

Databricks is a unified analytics platform built on Apache Spark. While GCP offers individual services (BigQuery, Dataproc, Dataflow), Databricks provides an **integrated lakehouse platform** that combines the best of data lakes and data warehouses. **Key Philosophy Difference:** • **GCP:** Best-of-breed individual services (BigQuery for DW, Dataproc for Spark, GCS for storage) • **Databricks:** Unified platform for all analytics workloads on a single architecture **Databricks runs ON cloud providers:** You can run Databricks on GCP, AWS, or Azure - it's cloud-agnostic.

1. ARCHITECTURE COMPARISON

Component	GCP Native	Databricks on GCP
Storage	GCS (object storage)	GCS (same) + Delta Lake format
Compute	Dataproc (Spark clusters) Dataflow (streaming)	Databricks clusters (optimized Spark + Photon)
Data Warehouse	BigQuery (columnar, serverless SQL)	Databricks SQL (Lakehouse SQL engine)
Catalog	Dataplex / BigQuery metadata	Unity Catalog (unified governance)
Orchestration	Cloud Composer (Airflow)	Databricks Workflows + Airflow support
Notebooks	Vertex AI Workbench Colab Enterprise	Databricks Notebooks (collaborative)

2. DATA PROCESSING COMPARISON

A. Batch Processing

Aspect	GCP Dataproc	Databricks
Setup	Create cluster manually or use serverless	Pre-configured optimized clusters
Startup Time	~5 minutes (cluster) ~30 sec (serverless)	~30 seconds (faster than Dataproc)
Optimization	Manual Spark tuning	Auto-optimization (AQE) Photon engine (2-10x faster)
Cost Model	Per-node-hour + GCP fee	Per-DBU-hour (higher but more efficient)
Management	More DIY	More managed
Use Case	Cost-sensitive, full Spark control	Developer productivity, performance

B. Streaming Processing

Feature	GCP Dataflow	Databricks Streaming
Model	Apache Beam (batch + streaming unified)	Spark Structured Streaming
Language	Java, Python, Go	Python, Scala, SQL
Exactly-once	Native support	Native support
Windowing	Beam windowing	Watermarks + windows
State Management	Beam state API	Spark streaming state
Auto-scaling	Automatic	Automatic
Late Data	Advanced windowing	Watermarks

3. DATA WAREHOUSING

Aspect	BigQuery	Databricks SQL
Architecture	Serverless, columnar MPP	Lakehouse (Delta Lake + SQL engine)
Storage	Proprietary columnar	Delta Lake on GCS (open format)
Pricing	Pay per TB scanned	Pay per DBU-hour (cluster)
Performance	Very fast for SQL	Fast (Photon engine)
Streaming Inserts	Native support	Via Delta Live Tables
Time Travel	7 days (up to 90)	Unlimited (Delta)
ACID	Yes	Yes (Delta Lake)
File Format	Proprietary	Open (Parquet-based)
Vendor Lock-in	High	Low (portable Delta)

Key Insight:
BigQuery excels at ad-hoc SQL analytics with zero management. Databricks SQL provides similar capabilities but with the advantage of unified governance across all workloads (batch, streaming, ML, SQL) using Delta Lake's open format.

4. DELTA LAKE (The Game Changer)

What is Delta Lake?

ACID transaction layer on top of Parquet files in GCS. Provides database-like guarantees on data lakes.

Feature	GCS + Parquet (Traditional)	GCS + Delta Lake
ACID Transactions	■ No	✓ Yes (transaction log)
Time Travel	■ Manual versioning	✓ Built-in (query old versions)
Schema Evolution	■ Break pipelines	✓ Safe additions/modifications
Upserts/Deletes	■ Rewrite partitions	✓ Efficient MERGE operation
Concurrent Writes	■ Conflicts	✓ Optimistic concurrency
Data Quality	■ Manual checks	✓ Constraints + expectations
Small Files	■ Performance issue	✓ Auto-compaction (OPTIMIZE)
Z-Ordering	■ Not available	✓ Multi-dimensional clustering

How Delta Lake Works:

- **Transaction Log (_delta_log):** JSON files tracking all changes (adds, deletes, metadata)
- **Parquet Data Files:** Actual data stored as Parquet (open format)
- **ACID via Optimistic Concurrency:** Multiple writers coordinate via transaction log
- **Time Travel:** Query any historical version using version number or timestamp
- **OPTIMIZE:** Compacts small files into optimal size (128MB-1GB)
- **Z-ORDER:** Colocates related data for faster queries (multi-column clustering)

GCP Translation:
Delta Lake provides BigQuery-like ACID guarantees on GCS data lakes. Think of it as making your GCS data lake queryable and transactional like BigQuery, but with open formats and lower storage costs.

5. DELTA LIVE TABLES (DLT)

What is DLT?

Declarative ETL framework. You define WHAT you want, DLT handles HOW to execute it.

Aspect	Cloud Composer (Airflow)	Delta Live Tables
Paradigm	Imperative (write steps)	Declarative (define outcome)
Dependencies	Manual (task >> task)	Automatic (inferred from queries)
Data Quality	Custom Python checks	Built-in expectations (WARN/DROP/FAIL)
Incremental	Manual watermarks	Automatic (streaming tables)
Error Handling	Manual retries	Auto-recovery
Monitoring	Custom dashboards	Built-in lineage + metrics
Code	Python DAGs	SQL or Python (simpler)
Best For	Complex orchestration across systems	ETL within Databricks data quality critical

DLT Key Features:

- **Expectations (Data Quality):** @expect("valid_amount", "amount > 0") - violations logged/dropped/failed
- **Streaming Tables:** @dlt.table() for batch, @dlt.streaming_table() for real-time
- **APPLY CHANGES:** Automatic CDC handling (merges inserts/updates/deletes)
- **Automatic Lineage:** Visual DAG of dependencies
- **Auto-scaling:** DLT manages cluster resources
- **Quarantine:** Bad data isolated, doesn't break pipeline

When to Use DLT vs Airflow:
Use DLT: Data transformations within Databricks, data quality critical, want simplicity
Use Airflow: Orchestrating across multiple systems (GCS, BigQuery, APIs, external DBs), complex conditional logic

6. UNITY CATALOG (Governance)

Feature	Dataplex / BigQuery	Unity Catalog
Scope	GCP services only	All Databricks workloads (batch, streaming, ML, SQL)
Access Control	IAM (coarse-grained)	Fine-grained (table/row/column)
Lineage	Dataplex lineage (limited)	Automatic end-to-end (across all workloads)
Audit	Cloud Audit Logs	Built-in audit logs (who accessed what, when)
Data Sharing	Manual	Delta Sharing protocol (secure cross-org sharing)
Catalog Model	Project > Dataset > Table	Metastore > Catalog > Schema > Table

Unity Catalog Hierarchy:

Metastore (top-level) → Catalog (like database) → Schema (like dataset) → Table/View

Advanced Features:

- **Row-level security:** Filter rows based on user identity
- **Column masking:** Redact sensitive columns (e.g., mask SSN for certain users)
- **Delta Sharing:** Share data with external organizations securely (no copying)
- **Automatic lineage:** Track data from source to dashboard

7. COST COMPARISON

A. Batch Processing Cost

GCP Dataproc: • Compute: $0.01/vCPU-hour + GCE instance cost • Example: n1-standard-4 (4 vCPU, 15 GB) = $0.19/hr + $0.04 Dataproc fee = $0.23/hr • 10-node cluster, 8 hrs/day = $18.40/day **Databricks on GCP:** • Compute: GCE instance cost + DBU cost • Example: Same n1-standard-4 = $0.19/hr + $0.40 DBU = $0.59/hr • 10-node cluster, 8 hrs/day = $47.20/day **BUT:** Databricks jobs often complete 2-5x faster due to Photon engine and auto-optimization. • If job completes in 3 hrs instead of 8 hrs: $0.59 × 3 × 10 = $17.70 (cheaper than Dataproc!)

B. Data Warehouse Cost

BigQuery: • Storage: $0.02/GB/month (active), $0.01/GB (long-term) • Query: $5/TB scanned (on-demand) or flat-rate slots **Databricks SQL:** • Storage: GCS cost ($0.02/GB/month) + Delta metadata overhead • Query: SQL warehouse cost (DBU-based, runs continuously or auto-stop) • Serverless SQL: Pay per query (similar to BigQuery) **Winner:** BigQuery for ad-hoc analytics, Databricks for unified lakehouse (single platform for all workloads)

8. DECISION MATRIX: When to Use What

Use Case	Use GCP Native	Use Databricks
Ad-hoc SQL analytics	✓ BigQuery (fastest, serverless)	Databricks SQL (if already on platform)
Batch ETL (simple)	✓ Dataproc Serverless (cheapest)	Databricks (if team productivity > cost)
Batch ETL (complex)	Dataproc (full Spark control)	✓ Databricks (auto-optimization, easier)
Streaming ETL	Dataflow (Beam-based)	✓ Databricks Streaming (Spark-based, unified)
Data Governance	Dataplex (GCP-wide)	✓ Unity Catalog (fine-grained, better lineage)
ML Workloads	Vertex AI	✓ Databricks ML (if data already in Delta)
Unified Platform	■ Multiple services	✓ Databricks (single platform)
Cost-sensitive	✓ GCP Native (cheaper infra)	Databricks (faster = less runtime)

General Recommendation:

Use GCP Native if: You're happy with BigQuery, want lowest infrastructure cost, prefer best-of-breed services **Use Databricks if:** You want unified platform, need advanced governance (Unity Catalog), team productivity matters more than infrastructure cost, want to avoid vendor lock-in (Delta Lake is open) **Hybrid Approach:** Many companies use BigQuery for analytics + Databricks for ETL/ML (write to Delta, also sync to BigQuery)

9. MIGRATION PATH: GCP → Databricks

1. **Start with Storage:** Keep data in GCS, add Delta Lake format on top
2. **Migrate Spark Jobs:** Dataproc → Databricks (mostly code compatible)
3. **Migrate Pipelines:** Airflow DAGs → Delta Live Tables (for Databricks-only pipelines)
4. **Governance:** Implement Unity Catalog (fine-grained access control)
5. **Analytics:** Keep BigQuery or migrate to Databricks SQL
6. **ML Workloads:** Vertex AI → Databricks ML (if data in Delta)

Key Translation for Your Resume:

• Your **Cloud Composer experience** → Databricks Workflows (or continue using Airflow)

• Your **BigQuery transformations** → Databricks SQL or Delta Live Tables

• Your **Dataproc jobs** → Databricks Spark jobs (mostly compatible)

• Your **GCS data lake** → Add Delta Lake format for ACID guarantees

• Your **Dataplex governance** → Unity Catalog (more fine-grained)

10. INTERVIEW ANSWER TEMPLATES

Question	Answer Template
Why Databricks over GCP native?	Databricks provides a unified lakehouse platform vs GCP's best-of-breed services. The advantage i
How is Delta Lake different from BigQuery?	BigQuery is a proprietary data warehouse with great performance but vendor lock-in. Delta Lake pr
DLT vs Airflow?	Delta Live Tables is declarative - you define the outcome, DLT handles execution. Airflow is impera
Can you run Databricks on GCP?	Yes, Databricks is cloud-agnostic. It runs on GCP infrastructure (GCE for compute, GCS for storage

Summary:

Databricks on GCP gives you the best of both worlds: GCP's infrastructure + Databricks' unified platform. Your GCP experience translates well - the underlying concepts (Spark, distributed computing, cloud storage) are identical. The main additions are Delta Lake (ACID on data lakes), DLT (declarative pipelines), and Unity Catalog (fine-grained governance).